

ReDas: A Lightweight Architecture for Supporting Fine-Grained Reshaping and Multiple Dataflows on Systolic Array

Meng Han¹, Liang Wang¹, Limin Xiao¹, Tianhao Cai¹, Zeyu Wang¹,
Xiangrong Xu¹, and Chenhao Zhang¹

Abstract—The systolic accelerator is one of the premier architectural choices for DNN acceleration. However, the conventional systolic architecture suffers from low PE utilization due to the mismatch between the fixed array and diverse DNN workloads. Recent studies have proposed flexible systolic array architectures to adapt to DNN models. However, these designs support only coarse-grained reshaping or significantly increase hardware overhead. In this study, we propose ReDas, a flexible and lightweight systolic array that supports dynamic fine-grained reshaping and multiple dataflows. First, ReDas integrates lightweight and reconfigurable roundabout data paths, which achieve fine-grained reshaping using only short connections between adjacent PEs. Second, we redesign the PE microarchitecture and integrate a set of multi-mode data buffers around the array. The PE structure enables additional data bypassing and flexible data switching. Simultaneously, the multi-mode buffers facilitate fine-grained reallocation of on-chip memory resources, adapting to various dataflow requirements. ReDas can dynamically reconfigure to up to 129 different logical shapes and 3 dataflows for a 128×128 array. Finally, we propose an efficient mapper to generate appropriate configurations for each layer of DNN workloads. Compared to the conventional systolic array, ReDas can achieve about $4.6\times$ speedup and $8.3\times$ energy-delay product (EDP) reduction.

Index Terms—Systolic array, DNN acceleration, fine-grained reshaping, multiple dataflows, reconfigurable roundabout data paths.

I. INTRODUCTION

DEEP neural networks (DNNs) have demonstrated remarkable accuracy across a wide range of tasks. As AI-powered applications continue to advance, various fields

such as autonomous driving, augmented reality (AR), virtual reality (VR), etc., are leveraging multiple deep neural networks to address diverse sub-tasks. The DNN model is constructed using a series of layers. Depending on the network topologies and structures of layers, different types of DNNs can be built such as Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), Multilayer Perceptrons (MLPs), Transformers, etc. Additionally, several variants of layers have been proposed to enhance the DNN. For instance, variants of convolutional layers [1] include 3D convolution, depth-wise convolution, deformable convolution, etc.

The significant heterogeneity in operations and shapes within and across DNN models leads to diverse workloads, posing a severe challenge in efficiently designing domain-specific accelerators. Due to their computation and memory-intensive nature, domain-specific accelerators have been actively developed to enhance DNN performance and energy efficiency [2], [3], [4], [5]. Systolic array architecture [6] is one of the premier architectural choices for DNN acceleration. Benefiting from the regular 2D processing element (PE) array structure, systolic array inherently exploits the spatial data reuse and computation parallelism for general matrix multiplication (GEMM), which is the key operation of DNNs [7]. However, fixed systolic arrays often exhibit low PE utilization under irregular workloads, indicating significant potential for architectural improvement. For many DNN layers such as Long Short-Term Memory (LSTM) and depth-wise convolution layer, the PE utilization can drop to less than 10% [8], [9], keeping the vast majority of PEs idle.

The fundamental cause of the low PE utilization comes from the mismatch between the fixed array and diverse DNN workloads. The shape (height and width of a 2D array) and dataflow are the two key properties of a systolic array. A promising solution should be capable of adapting to diverse DNN workloads by dynamically reconfiguring both the topological shape and the dataflow with a limited hardware overhead. We have observed that if the shape and the dataflow of an array can ideally adapt to DNN models layer by layer, it can achieve more than $6.3\times$ speedup against a 128×128 fixed architecture for EfficientNet-B0 [10] (detailed in Section II-C).

Recent studies have proposed flexible systolic array architectures to accommodate various DNN models. Unfortunately, prior works are still far from achieving this goal. Gemmini [11]

Manuscript received 18 July 2023; revised 1 April 2024; accepted 19 April 2024. Date of publication 9 May 2024; date of current version 11 July 2024. This work was supported in part by the National Key R&D Program of China under Grant 2023YFB4503100, in part by the National Natural Science Foundation of China under Grant 62272026 and Grant 62104014, in part by Beihang Frontier Interdisciplinary Fund under Grant YWF-23-Q-1015, in part by the Academic Excellence Foundation of BUAA for Ph.D. Students, and in part by the Iluvatar CoreX Semiconductor Company, Ltd. Recommended for acceptance by P. Milder. (Corresponding authors: Liang Wang; Limin Xiao.)

The authors are with the State Key Laboratory of Complex & Critical Software Environment (CCSE) and School of Computer Science and Engineering, Beihang University, Beijing 100191, China (e-mail: hanm@buaa.edu.cn; lwang20@buaa.edu.cn; xiaolm@buaa.edu.cn; caitianhao@buaa.edu.cn; wangzy1002@buaa.edu.cn; xxr0930@buaa.edu.cn; zch13021728086@buaa.edu.cn).

Digital Object Identifier 10.1109/TC.2024.3398500

designs a flexible PE structure that supports both Output Stationary (OS) and Weight Stationary (WS) dataflows while the shape of the systolic array remains fixed. Planaria [12] enables coarse-grained reshaping of the systolic array to five different logical shapes under WS dataflow. Similarly, DyNNamic [13] can reshape the systolic array to various logical shapes under OS dataflow. However, both Planaria and DyNNamic only support a specific dataflow, which limits their efficiency in accommodating diverse DNN workloads. SARA [14] is a recent work that allows for reconfiguration of both the systolic array shape and the dataflow. However, the dense dedicated links and multi-port buffer components of SARA incur significant overhead in terms of area and energy. Therefore, it is necessary to develop a flexible systolic array architecture that supports fine-grained reshaping and multiple dataflows, accommodating various DNN models while maintaining low design overhead.

In this paper, we present ReDas, a flexible and lightweight systolic array architecture that supports fine-grained reshaping and multiple dataflows. ReDas can dynamically reshape to up to 129 different logical shapes and 3 dataflows for a 128×128 PE array. To achieve this, we first propose reconfigurable roundabout data paths, which achieve fine-grained reshaping using only short connections between adjacent PEs and enable the data movement along two dimensions. Second, we design the microarchitecture of the PE and multi-mode data buffer. The PE structure introduces additional data bypassing and flexible data switching, which allow each PE to work at arbitrary dataflows and deal with the data from four directions. The multi-mode buffer supports fine-grained re-allocation of the on-chip memory resources to adapt to the requirement of different dataflows. Finally, to maximize the benefits of ReDas, we propose a mapping strategy that allows ReDas to adapt to various DNN models in a layer-by-layer manner. We have developed an elaborate analytical model to estimate the performance of ReDas under specific configurations. This analytical model takes into account additional constraints such as on-chip buffer capacity, off-chip bandwidth, as well as the ping-pong work mode. By considering these factors, we can more accurately identify the appropriate logical shape and dataflow for ReDas, ensuring efficient and effective utilization of the architecture.

We implement ReDas and evaluate it in eight DNN models. The results demonstrate that ReDas outperforms conventional systolic array with about $4.6\times$ speedup and $8.3\times$ energy-delay product (EDP), respectively. Compared to SARA, ReDas achieves about $2.1\times$, $2.3\times$ and $3.5\times$ improvement in terms of power efficiency, energy-delay product (EDP) and area-delay-product (ADP), respectively.

To that end, the paper makes the following contributions:

- We introduce a lightweight and reconfigurable roundabout data path using only the short connections between neighbor PEs. Compared to the dedicated bypass data path, the shared roundabout data path shows better scalability and lower overhead.
- We present an efficient systolic architecture named ReDas by leveraging the reconfigurable roundabout data paths. By allowing the data movement along two dimensions, ReDas can flexibly support fine-grained reshaping and multiple dataflows.

- We propose ReDas Mapper to adapt to various DNN models. The mapper employs an elaborate analytical model and interval sampling to search the suitable hardware configuration and workload mapping.

The rest of the paper is organized as follows. In Section II, we introduce the necessary background of systolic array-based accelerators, related works and the core motivation of this work. In Section III, we describe the proposed architecture in detail. In Section IV, we explain the mapping strategy for ReDas to adapt to various DNN models. In Section V, we present the experimental methodology and evaluation result. Finally, in Section VI, we conclude this article.

II. BACKGROUND AND MOTIVATION

A. Heterogeneous DNN workloads

Currently, the integration of multiple DNNs within AI systems is becoming increasingly prevalent. DNNs are constructed using a series of layers. Each type of layer comprises a set of parameters and contains multiple variants, resulting in highly heterogeneous workloads in AI systems. For instance, Baidu Apollo autonomous system employs multiple DNN models in the sub-tasks of perception and prediction stages to enhance performance, such as traffic light detection, line detection, semantic segmentation and trajectory prediction [15]. The DNN workloads of the system involve various operations such as CONV2D, LSTM, MLP, Fully Connected (FC), and Depth-wise Convolution (DWCONV).

While there are various DNN layers, general matrix multiplication (GEMM) remains the preferred abstraction. Most DNN layers can be transformed into GEMMs. In CNN, the convolutional layer can be transformed into a GEMM by the *im2col* algorithm [16], which unrolls high dimensional tensors into matrices. The FC layer is a GEMM without any transformation. In RNN, the LSTM layer contains 8 matrix-vector multiplications. The matrix-vector multiplication is a special case of GEMM. In the Transformer, the Multi-Head Attention (MHA) layer involves different numbers of GEMMs, depending on the number of heads of an MHA. The layer transformation process can be done during the DNN compilation phase to simplify hardware design to GEMM acceleration. Besides those linear layers, the DNN models involve non-linear layers (e.g., Rectified Linear Unit (ReLU), softmax, sigmoid, tanh), which can not be transformed into GEMM, and they are treated as vector operations [17].

Prior works [7], [18], [19] have shown that the GEMM operation is the performance bottleneck operation for DNN workloads. GEMM operations account for approximately 70% of the total runtime during training [7] and even higher during inference. The heterogeneous nature of DNN workloads introduces both regular and irregular GEMM operations [7]. For example, in Resnet-50 [20], the dimensions (M, N, K) of GEMM operations can vary widely, with a total of 21 different combinations. These dimensions can range from (49, 2048, 512) to (12544, 147, 64). As a result, efficiently supporting heterogeneous GEMM workloads has become a crucial consideration in the design of DNN accelerators.

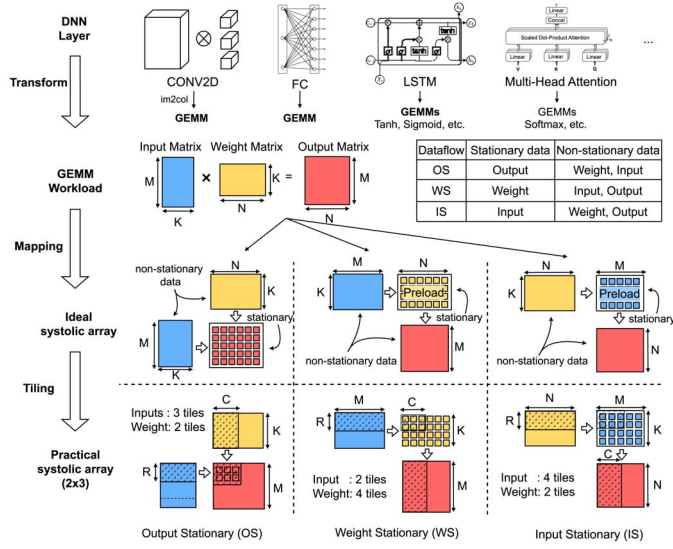


Fig. 1. Illustration of the execution process in the systolic array with different dataflows.

B. Systolic Arrays

Systolic array is a premier architectural choice for accelerating DNN workloads. Systolic array architectures consist of two-dimensional arrays of PEs interconnected by peer-to-peer links. These architectures effectively exploit spatial data reuse and computation parallelism by enabling data transfer between neighbor PEs within the same rows and columns. Fig. 1 provides an illustration of the conventional systolic array execution for DNN models. In this execution, a DNN layer such as CONV2D, FC, LSTM, and MLP is transformed into one or more GEMM operations. A GEMM operation involves multiplying an input matrix of size $M \times K$ with a weight matrix of size $K \times N$, resulting in an output matrix of size $M \times N$.

Dataflow and shape are the two key properties of the systolic array. There are three major kinds of dataflows for systolic array called Output Stationary (OS), Weight Stationary (WS) and Input Stationary (IS) [3]. Each dataflow maps one matrix (named stationary data) to the PE array while the remaining two matrices (named non-stationary data) are transferred cycle by cycle through the PE array in horizontal and vertical directions, respectively. The shape of a systolic array is another major property for GEMM execution. Designing a large enough systolic array which can map all the computing at once is costly. Practically, when the matrix size exceeds the practical systolic array shape, the matrix is divided into multiple tiles. Each tile will then be mapped onto the array sequentially until all outputs are generated. Fig. 1 gives an example that the input matrix and weight matrix are split into multiple tiles under OS, WS, IS dataflows for a 2×3 systolic array.

C. Diverse Dataflow and Shape Requirements

While the systolic array architecture inherently exploits spatial data reuse and computation parallelism for DNN workloads, it still faces challenges in efficiently handling diverse DNNs which result in low PE utilization [9]. This issue is caused by

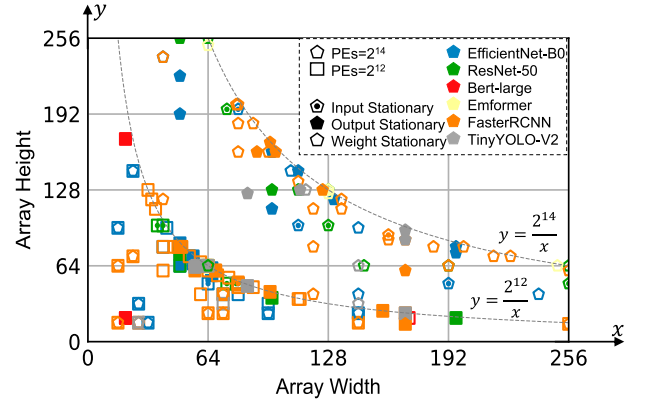


Fig. 2. Ideal dataflow and physical shape of systolic array vary with each layer of DNN models. Total number of PEs is not greater than 2^{12} or 2^{14} .

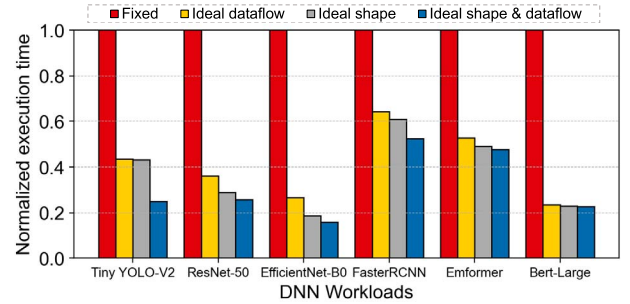


Fig. 3. Normalized execution time under different situation. **Fixed:** a 128×128 fixed PE array with WS dataflow. **Ideal dataflow:** the dataflow (WS/OS/IS) of PE array is assumed to optimally adapt to DNN models layer per layer, the shape is fixed as 128×128 . **Ideal shape:** the shape of PE array is assumed to optimally adapt to DNN models layer per layer, and the total number of PE is not greater than 128×128 . The dataflow is fixed as WS. **Ideal shape & dataflow:** the shape and dataflow of PE array are assumed to optimally adapt to DNN models layer per layer.

the significant variation in computational characteristics, such as channel size and filter size, across different networks or even layers within single DNN model. Consequently, there is no one-size-fits-all configuration for a systolic array that achieves the best performance across all DNN workloads.

As shown in Fig. 2, we observe that the ideal¹ array size (height and width of the physical shape) and dataflow (WS/OS/IS) vary from DNN layer to layer. To determine these ideal configurations, we explore all combinations of array shapes and dataflows within the constraints of the total number of PEs ($\leq 2^{12}$ or 2^{14}) and available dataflows. The distribution of these configurations demonstrates a significant dispersion, highlighting the need for a flexible systolic array to support fine-grained reshaping and multiple dataflows.

As shown in Fig. 3, another case study is conducted to show the significant potential of the flexible systolic array. We evaluate the execution time for running various DNN models in four situations. The case study shows that if the shape and the dataflow of an array can be ideally adapt to DNN models

¹The “ideal” refers to the configuration that achieves the minimum execution time for the specific DNN layer.

TABLE I
THE COMPARISON OF FLEXIBLE SYSTOLIC ARRAY
BASED DNN ACCELERATORS

	Multiple Dataflows	Fine-Grained Reshaping	Low Wire Overhead	Low Buffer Overhead
Gemmini [11]	✓		✓	✓
Planaria [12]			✓	✓
DyNNamic [13]		✓		
SARA [14]	✓	✓		
ReDas (ours)	✓	✓	✓	✓

layer by layer, it can achieve more than $6.3\times$ speedup against a 128×128 fixed architecture for EfficientNet-B0. Moreover, the improvement is reduced if only one latitude of reconfiguration is supported.

D. Related Works

The heterogeneous nature of DNN workloads has driven the development of more flexible architectures. Several studies have proposed novel architectures that allow for dynamic reconfiguration of systolic array's dataflow and logical shape. Table I provides an overview of these studies. Gemmini [11] and Planaria [12] support multiple dataflows or coarse-grained reshaping of the systolic array. On the other hand, DyNNamic [13] and SARA [14] support fine-grained reshaping along with one or multiple dataflows.

Gemmini introduces a flexible PE structure that supports both OS and WS dataflows, while keeping the shape of the systolic array fixed. Planaria breaks up the systolic array into multiple 32×32 sub-arrays and designs bypassing data paths between these sub-arrays to enable reshaping. However, Planaria only supports coarse-grained reshaping with a limited set of 5 logical shapes (without partitioning).

DyNNamic divides the convolution operations into shared kernels convolution and weighted accumulation stages, and designs a deformable systolic array under OS dataflow. It vertically splits the systolic array into sub-arrays and introduces additional bypassing data paths for inter-sub-array communication. Similar to Planaria, DyNNamic supports fine-grained reshaping under specific dataflow, limiting its flexibility for diverse DNN workloads.

SARA is a recent work that offers enhanced flexibility in both the systolic array shape and dataflow. It divides the PE array into multiple 4×4 sub-array and allocates dedicated links from on-chip buffer to the edge of each sub-array in both directions. Specially, the buffers are multi-ported with each bypass link allocated to separate ports. While the dedicated bypass links and multi-ported buffers of SARA enhance flexibility, they introduce significant overhead in wire usage, energy consumption, and power cost, leading to considerable increases in both power and area requirements.

E. Overhead of Flexibility

Recent studies have explored diverse mechanisms to enhance flexibility in dataflow and reshaping capabilities for DNN

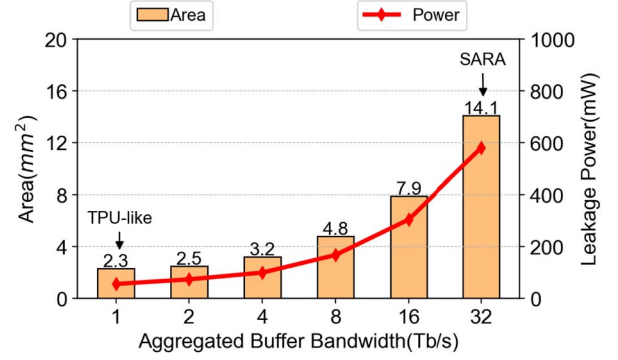


Fig. 4. The area and leakage power of 1MB multi-ported buffer under varying aggregated buffer bandwidth conditions. Assume the clock frequency is 1GHz.

accelerators. SARA and DyNNamic employed additional bypass links and multi-ported buffers to support fine-grained reshaping. However, these components significantly increase buffer and wire usage, thereby raising area and power consumption.

Wire Cost: Current designs often incorporate additional bypass links within the systolic array to enable dynamic reshaping of its logical shape. These bypass links play a crucial role in facilitating data movement among non-adjacent PEs and on-chip buffers. However, the interconnects pose significant challenges in the physical design process. We have conducted a place-and-route experiment for Gemmini-style and SARA-style PE array. The report reveals that the total wire length of a 32×32 SARA-style PE array is approximately $9.5\times$ longer compared to a Gemmini-style PE array. The bypass links are the predominant contributor to the extra wire length. The increased wire length results in higher capacitance and resistance, leading to increased delay in a quadratic trend [21]. In addition, numerous BUFs are inserted within the interconnects to meet timing constraints [22], causing additional area and power overhead.

Buffer Cost: SARA and DyNNamic utilize multi-ported buffers to address bandwidth limitations and optimize buffer capacity utilization. However, the multi-ported buffer comes with significant area and power costs.

DyNNamic utilizes multi-ported SRAM to construct its multi-ported buffer. However, due to the modifications made to the SRAM bit cell, the area growth exhibits a quadratic relationship with the number of ports [23]. As a result, scaling up DyNNamic from a 20×20 size to a larger size, such as 100×100 , would necessitate the use of 20-ports SRAM, posing practical challenges and limitations.

SARA provides dedicated SRAMs for each 4×4 sub-array in both directions. In particular, 1024 1KB SRAMs are employed to construct a 1MB 1024-ported buffer. Although such design is able to provide $32\times$ bandwidth higher than TPU-like buffer, the overhead is significant. To quantify the area and power costs, we have conducted a case study following the buffer configuration in SARA. Several SRAMs with varying depths are employed to emulate a 1MB buffer under different aggregated bandwidth conditions. These SRAMs are generated by TSMC 28nm memory compiler. Fig. 4 shows that to meet

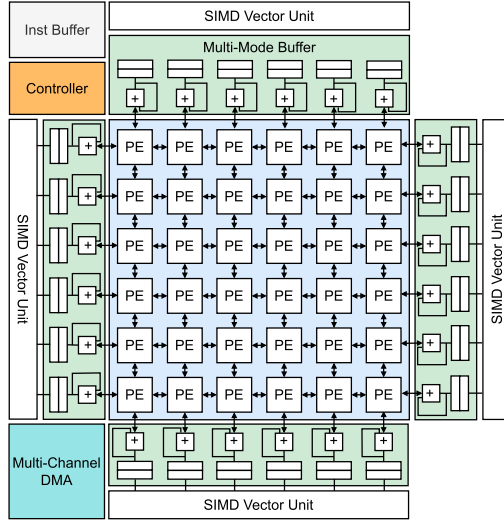


Fig. 5. The overall architecture of ReDAS.

the bandwidth requirement of SARA, the buffer area increases from 2.3 mm^2 to 14.1 mm^2 , and the leakage power increases from 56 mW to 580 mW, significantly impacting the overall chip cost.

F. Design Consideration

The heterogeneous nature of DNN workloads demands a more flexible architecture. Numerous studies have introduced novel architectures enabling dynamic reconfiguration of systolic arrays in terms of dataflow and shape. However, these architectures often suffer from undesirable impact on area and power consumption.

To enhance efficiency and cost-effectiveness, exploring new systolic array architectures that support multiple dataflows and fine-grained reshaping is crucial. Such architecture should prioritize short connections among adjacent PEs to minimize the design cost. Additionally, it should take full advantage of the data reuse among PEs to reduce the overall bandwidth requirements for the on-chip buffer.

III. FINE-GRAINED RESHAPING AND MULTIPLE DATAFLOWS FOR SYSTOLIC ARRAY

A. Architecture Overview

We present ReDAS, a flexible and lightweight systolic array architecture that supports dynamic fine-grained reshaping and multiple dataflows. The overall architecture of the proposed accelerator is illustrated in Fig. 5, which consists of four types of components, 1) a PE array, 2) four multi-mode buffers surrounding the PE array, 3) the SIMD vector units near the multi-mode buffers, and 4) other components, such as an instruction buffer, control unit, and DMA.

In the PE array, the neighboring PEs are interconnected using bidirectional links. These links enable the dynamic establishment of the roundabout data path, which is the key for supporting fine-grained reshaping of the systolic array.

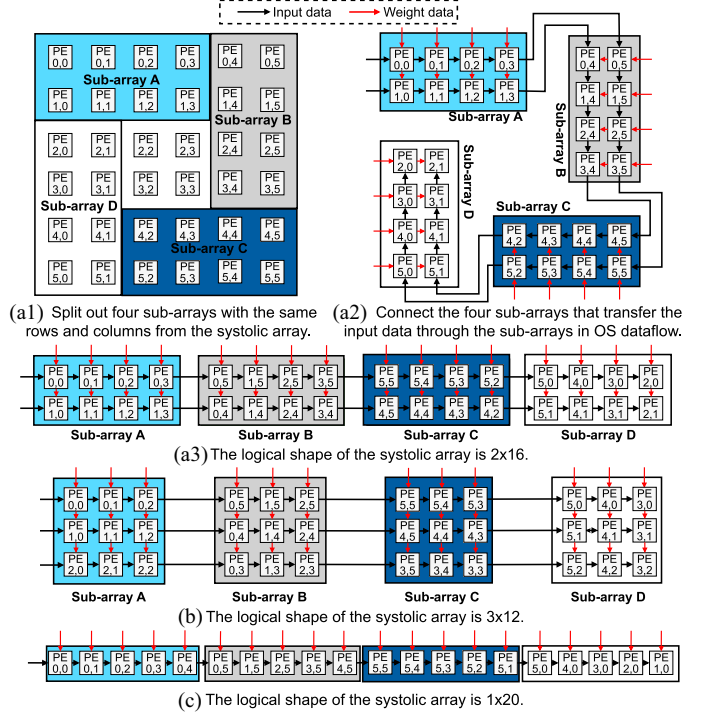


Fig. 6. Illustration of ReDAS execution on OS dataflow.

The multi-mode buffers connect to both the multi-channel off-chip DRAM and the PE array. Specifically, the multi-mode buffers consist of multiple banks, with each bank serving different roles depending on the dataflows and logical shapes being utilized. To support WS and IS dataflows, the accumulators are integrated in the multi-mode buffer.

ReDAS employs four SIMD vector units to execute non-linear operations. In the runtime, the PE array and SIMD units work in a pipeline manner [11]. ReDAS integrates NN-LUT [24] into SIMD units. NN-LUT is an accurate and hardware-friendly architecture that efficiently performs various operations (e.g., tanh, sigmoid, exponent, etc.) of the non-linear layers.

B. Fine-Grained Reshaping With Roundabout Data Path

The proposed systolic array can dynamically reshape into a number of logical shapes. Fig. 6 shows an example where a 6×6 systolic array is reconfigured into 3 of 7 possible logical shapes, which are 2×16 , 3×12 and 1×20 under OS dataflow. The key steps of reshaping into 2×16 are shown in Fig. 6(a1), (a2), (a3). In this case, the PE array is split into four sub-arrays, which are marked in different colors. And then, the sub-arrays are chained end-to-end using roundabout data paths, which enable the data movement along 2 dimensions. In this manner, the input data, marked as the black lines, is transferred from the header PE in Sub-array A (PE[0, 0] and PE[1, 0]) to the tail PE in Sub-array D (PE[2, 0] and PE[2, 1]). The weight data is issued to the PE array from four edges, marked as red lines. Fig. 6(a3) presents an equivalent logical shape with 2×16 size. Other shapes can also be configured using the similar steps.

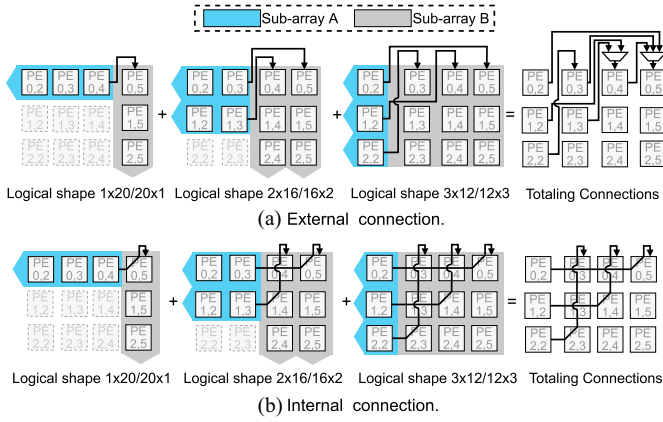


Fig. 7. Two implementations of roundabout data path between the sub-array A and sub-array B in 6x6 PE array. For better explanation, the figure only shows partial 6x6 PE array.

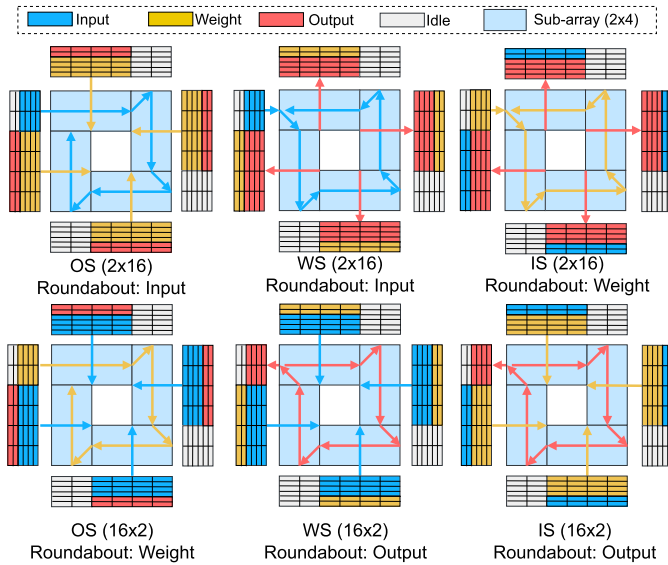


Fig. 8. Examples of the on-chip buffers' working modes varying with the dataflow and logical shape. The dataflow, logical shape, and data in the roundabout data paths are represented in the diagrams.

The roundabout data paths inherently support the fine-grained reshaping, as the sub-array can be allocated with various rows and columns of PEs. A logical shape is constructed by chaining 4 sub-arrays using the roundabout data paths. Assuming the size of each sub-array is $R_s \times C_s$ with OS dataflow, the corresponding logical shape is $R_s \times 4C_s$ if the input data is transferred along the roundabout data path, or $4C_s \times R_s$ if the weight data is transferred along the roundabout data path. Fig. 8 gives an example of reshaping to 2×16 and 16×2 via the sub-array sized 2×4 . Furthermore, to manage costs, the height of the sub-array should not exceed half of the physical PE array. Thus, a PE array with $R \times R$ size totally supports $R + 1$ different logical shapes. For example, a 6×6 array can be reshaped into 7 logical shapes of 1×20 , 20×1 , 2×16 , 16×2 , 3×12 , 12×3 and 6×6 .

Fig. 7 shows two possible implementations of the roundabout data paths, i.e., the external connection manner and the internal connection manner. The external connection method directly connects the source and destination. However, this method is only suitable for a small-scale systolic array. Since there are multiple sources with the same destination, MUX units are employed to switch the sources. Moreover, the inevitable long connections break the regular short connections in the original systolic array.

ReDas uses the internal connection manner for the roundabout data paths as depicted in Fig. 7(b). The internal manner can achieve a higher scalability and lower design cost. This is because the additional connections are only established among the adjacent PEs and are reusable under different logical shapes. The detailed design of the PE that supports internal connection manner is presented in Section III-D. Note that although the roundabout data paths may not use all the PEs in the systolic array, ReDas can still greatly improve the PE utilization up to 4 times in some cases compared to the conventional systolic array. The detailed discussion is in Section V-H.

C. Multiple Dataflows With Multi-Mode Buffer

ReDas efficiently supports multiple dataflows by leveraging the symmetrical patterns inherent in these dataflows. As discussed in Section II-B, for each dataflow, the PEs receive two non-stationary data and perform the Multiplier-and-Accumulation (MAC) operation with the stationary data scratched in the PE register. The PE can also flexibly adapt to three calculation patterns by adding an exchanging logic.

Unlike the conventional systolic array architecture, ReDas employs four on-chip buffers called multi-mode buffers, which are positioned around the PE array. Each buffer consists of multiple independent banks. Such design easily supports dynamic fine-grained memory resource reallocation. In addition, to support WS and IS dataflow, the additional accumulators are integrated within each buffer (OS dataflow does not need accumulators).

The type of operand stored in the multi-mode buffer is variable. In detail, both the stationary data and non-stationary data can be stored in the same buffer. During runtime, each bank within the buffer plays a special role of weight issuer, input issuer, output receiver or idleness depending on the specific dataflows and logical shape. Fig. 8 illustrates the working modes of the buffers with different dataflows and logical sizes. The buffers are marked with four different colors representing the four working modes mentioned above. The solid arrows in the figure depict the movement of data. To optimize energy efficiency, when the buffer bank is set to an idle state, the SRAMs within the bank are switched to sleep mode for leakage power reduction.

In the WS/IS dataflow, the PE array preloads the stationary data (weight/input data) from the multi-mode buffer in each direction. The data is transferred from the edges of the PE array towards the center. Once the data is loaded, the GEMM calculation begins. On the other hand, in the OS dataflow, at the end of the GEMM computation, the result data in each PE

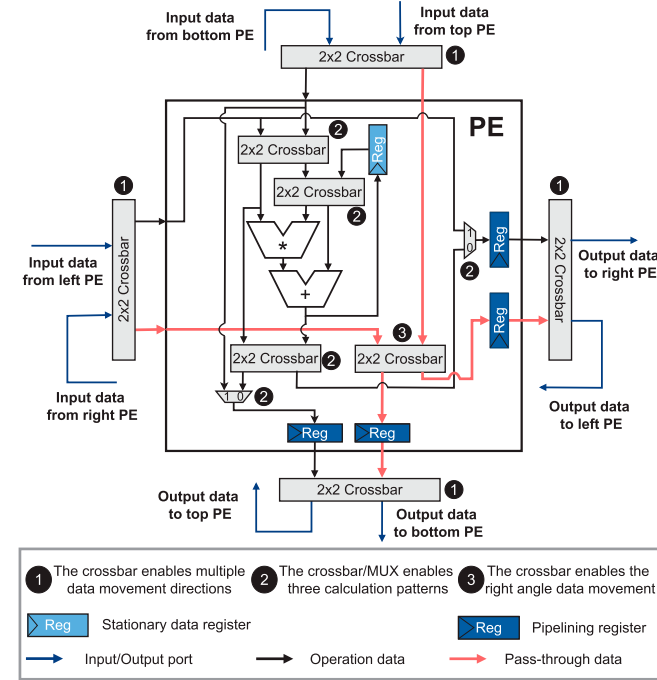


Fig. 9. Structure of the PE.

is transferred from the center to the edges of the PE array, and then it is stored back into the multi-mode buffers.

The fine-grained multi-mode buffer in ReDAS offers two significant benefits. First, the multi-mode buffer allows for flexible allocation of operand data, enabling ReDAS to meet the varying data requirements of different PE array logical shapes. For example, in the OS dataflow, when the PE array shape is 128×128 , the input data and weight data have equivalent storage requirements. However, when the PE array is reshaped to 64×256 , the input data requirement becomes four times that of the weight data. The multi-mode buffer can adapt to these changes and allocates the appropriate amount of buffer space, ensuring efficient utilization of resources. Second, the hybrid storage capability of the multi-mode buffer in ReDAS improves the utilization of buffer bandwidth. This is achieved by storing both stationary data and non-stationary data within the same buffer. As the preloading of stationary data stage and the computation stage do not occur simultaneously. This ensures that the buffer remains busy during both stages, maximizing its utilization without the need for multi-port SRAM design.

D. PE Structure

The internal connection manner and three calculation patterns require a more flexible PE structure. Fig. 9 shows the details of PE structure in our design. The MAC unit is used for the GEMM operation, as in the canonical PE. Furthermore, the ReDAS PE features four input ports and four output ports, evenly distributed around the PE. The bidirectional connections between neighbor PEs are established to support the roundabout data path.

The PE employs several crossbars and MUX units to support a variety of working modes. When a PE is employed to perform

TABLE II
IMPORTANT TERMS AND DESCRIPTIONS

Term	Description
R_p, C_p	The number of physical PE array rows and columns. ²
R_l, C_l	The number of logical PE array rows and columns.
<i>Dataflow</i>	The organization and movement of data within the PE array, including WS, OS and IS.
D_{phy}	The physical capacity of each multi-mode buffer bank.
D_{sta}, D_{non}	The allocated capacity of the multi-mode buffer bank for the stationary data tile and the nonstationary data tile.
M, K, N	The GEMM workload dimensions. Input activation matrix: $M \times K$; Weight matrix: $K \times N$; Output matrix: $M \times N$.
M_t, K_t, N_t	The tile dimensions. Input activation tile: $M_t \times K_t$; Weight tile: $K_t \times N_t$; Output tile: $M_t \times N_t$.
S_i, S_w, S_o	The data size of each type of tile. Input tile size $S_i = M_t \times K_t$; Weight tile size $S_w = K_t \times N_t$; Output tile size $S_o = M_t \times N_t$.
NUM_t	The number of iterations in which the PE array computes the tiled GEMM workload. $NUM_t = \lceil \frac{M}{M_t} \rceil \lceil \frac{K}{K_t} \rceil \lceil \frac{N}{N_t} \rceil$
$T_r(s), T_w(s)$	DRAM access time for reading or writing data of size s .

both the MAC operation and the roundabout data path (e.g., PE[0,4] and PE[1,3] of the logic shape 3×12 in Fig. 7(b)), the PE has up to four inputs simultaneously. Two of them (named operation data) are the non-stationary data, which are sent to the MAC unit, and the other two (named pass-through data) are on the roundabout data path. The crossbars (1) near the input ports differentiate the inputs into operation data and pass-through data. The pass-through data can either move straight or turn depending on the position of the current PE. The crossbar (2) enables the selection of the 2 data movements. For the operation data, to satisfy the special input order of the three operands of the MAC unit, the crossbars (2) above the MAC unit reorder these three operands. After the computation of MAC, the crossbar and MUX unit (2) below the MAC unit recover these data to the original order. Finally, the crossbars (1) near the output ports issue the pass-through data and operation data to destination ports.

IV. CONFIGURATION AND MAPPING

ReDAS's high flexibility in dynamic reshaping and dataflow switching requires an efficient hardware configuration and GEMM mapping method during model compilation, crucial for its overall performance. We propose ReDAS Mapper, a dedicated configuration and mapping engine to reach the best performance. ReDAS Mapper comprises three components: 1) a search space generator to create valid candidates of hardware configuration and GEMM mapping, 2) an analytical model for performance estimation, and 3) an interval sampling engine to efficiently reduce search space. Table II describes all the important terms used below.

For a given DNN model, in the compilation phase, the model is transformed into a sequence of GEMMs. Then, for each

²In this paper, we assume the physical PE array is square in shape, so that R_p is equal to C_p .

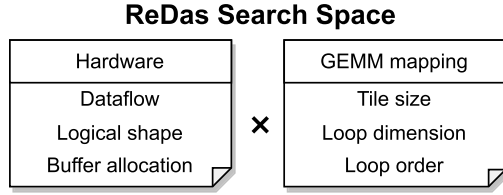


Fig. 10. The ReDas search space includes hardware configuration space and GEMM mapping space.

GEMM workload, ReDas Mapper generates the search space. Since the size of the search space can be extremely large (exceeding 10^{10} for a single GEMM workload), ReDas Mapper employs interval sampling to prune the candidates. Finally, it estimates the runtime of each candidate using the analytical model and chooses the configuration and mapping with the minimal runtime. In the execution phase, ReDas sequentially runs every GEMM workload of the model. At the start of each GEMM workload, ReDas takes 128 cycles to configure the PE array and buffers according to the configuration information from ReDas Mapper.

A. ReDas Search Space

As shown in Fig. 10, the ReDas search space is constructed from two orthogonal dimensions: 1) hardware configuration space, which includes logical array shape, dataflow, and buffer allocation; and 2) GEMM mapping space, encompassing tile size, loop dimension, and loop order. These dimensions significantly influence ReDas's runtime and power efficiency.

Logical shape and dataflow determine the actual behavior of the PE array and how ReDas processes the operand matrices. Equation (1) describes all the possible logical shapes for a ReDas array that supports PE-level reshaping granularity.

$$\begin{cases} 0 < R_l \leq \frac{1}{2}R_p \\ C_l = 4(C_p - R_l) \end{cases} \quad \text{or} \quad \begin{cases} 0 < C_l \leq \frac{1}{2}R_p \\ R_l = 4(R_p - C_l) \end{cases} \quad \text{or} \quad \begin{cases} R_l = R_p \\ C_l = C_p \end{cases} \quad (1)$$

These logical shapes and three dataflows (WS, OS and IS) jointly construct the PE array configuration space, which is the key of ReDas search space that significantly influences the performance on diverse GEMM workloads.

Buffer allocation dictates how the multi-mode buffer bank is shared between the stationary operand and one of two nonstationary operands. The buffer allocation can significantly affect on-chip data reuse and potentially impact the overall runtime. For each bank, the total allocated capacity for these operands cannot exceed its physical capacity. The constraint is defined as

$$D_{sta} + D_{non} \leq D_{phy} \quad (2)$$

Tile size is the actual operand matrices' dimensions (M_t , N_t and K_t) consumed by the PE array in each iteration. The valid tile size is constrained by the logical shape of ReDas array and the allocated buffer capacity. To maximize the PE utilization, ReDas Mapper sets two of the three dimensions (depending on the dataflow) equal to the logical array dimensions R_l and

C_l . The remaining dimension, however, has a direct influence on the first and last DRAM access overhead, which cannot be hidden by double buffering and becomes crucial on GEMM workloads that are not big enough to split into multiple tiles. It also has an impact on actual DRAM bandwidth utilization, which subsequently affects the runtime.

Loop dimension and order determine the tile access and execution order of a GEMM workload. A GEMM workload is divided into a certain amount of tiles of identical size. The access and execution order of these tiles greatly impacts the on-chip buffer utilization and data reuse rate, subsequently affecting the duplicate DRAM accesses and the runtime.

Owing to its high flexibility and the range of GEMM mapping choices, the ReDas search space can become extremely large if generated naively without any reduction. For example, a 128×128 ReDas can support 129 logical shapes and 3 dataflows, and the physical capacity of a buffer bank is 4096 words. On a GEMM workload of dimensions (784, 256, 128), the number of valid candidates in search space is over 5.7×10^{10} . To improve the search efficiency of ReDas Mapper without losing much performance, we employ an interval sampling engine, which is discussed in Section IV-C.

B. ReDas Analytical Model

The ReDas analytical model estimates the overall runtime for a given GEMM workload at a specific hardware configuration and GEMM mapping. Equation (3) describes the total runtime using a double buffering approach:

$$T_{total} = T_{start} + NUM_t \times \max\{T_{exe}, T_{rd\&wt}\} + T_{end} \quad (3)$$

where T_{start} is the time cost for reading the first input activation tile and weight tile from the off-chip DRAM and configuring the PE array, T_{exe} is for the systolic array to compute one tile, $T_{rd\&wt}$ is for off-chip DRAM reading and writing for one set of tiles in the case that there is no on-chip reuse to avoid the DRAM access, T_{end} is for writing the last output tile back to the off-chip DRAM.

Equation (4) represents the cycles cost for the systolic array to compute one tile with WS dataflow (each dataflow has a slightly different version of this equation).

$$T_{exe} = \begin{cases} R_l + (R_l + C_l + M_t - 1) + 4 \times R_l & R_l < C_l \\ C_l + (R_l + C_l + M_t - 1) + 4 \times C_l & R_l > C_l \\ R_l + (R_l + C_l + M_t - 1) & R_l = C_l \end{cases} \quad (4)$$

The execution cycle is comprised of three parts. The first term represents the cycles for preloading a weight tile from the multi-mode buffer to each PE, with data being transferred from the edges of the PE array towards the center. The second term ($R_l + C_l + M_t - 1$) is for consuming input activation tile and producing output tile. The third term ($4 \times R_l$ or $4 \times C_l$) is the additional bypass cycles incurred by the roundabout data path when the logical shape differs from the physical shape. When $R_l < C_l$, ReDas requires R_l cycles to rotate the flowing data by 90° at each of the four corners, adding a total $4 \times R_l$ to T_{exe} . When $R_l > C_l$, the additional bypass cycles amount to $4 \times C_l$. In the case where $R_l = C_l$, the PE array operates as a

conventional systolic array without reshaping, thus incurring no additional bypass costs.

Equation (5) describes the DRAM access cycles T_{start} , T_{end} and $T_{rd \& wt}$.

$$\begin{cases} T_{start} = \max\{T_r(S_i) + T_r(S_w), R_p\} \\ T_{end} = T_w(S_o) \\ T_{rd\&wt} = T_r(S_i) + T_r(S_w) + T_w(S_o) \end{cases} \quad (5)$$

The value of T_{start} is equal to the maximum of the data loading time and PE array configuration time. At the beginning of a GEMM workload execution, ReDas loads the initial input activation and weight tiles from off-chip memory. Simultaneously, it performs the configuration of the PE array from top to down, which takes R_p cycles. Because of the overlapping of these two operations, the hardware configuration barely causes extra cycles.

The analytical model takes on-chip tile reuse into consideration, which means the tiles already staged in the buffer do not need to be loaded again. This is implemented by a reuse-sensitive tile access sequence indicating whether each tile needs DRAM access, which is generated by ReDas Mapper before the runtime estimation.

In practice, it is observed that the actual DRAM access efficiency largely depends on the data volume in a single DMA transaction, and hence the maximum DRAM bandwidth is too ideal for runtime estimation directly. An approximation method is proposed for more accurate estimation. We prerecord the actual DRAM access latency when reading and writing different amounts of data, and approximate the latency for accessing data of given size by linear interpolation, which is what function $T_w(s)$ and $T_r(s)$ implement in detail.

C. Interval Sampling Engine

ReDas Mapper employs an interval sampling engine to effectively reduce the search space for faster mapping while remaining near minimal GEMM workload runtime.

ReDas Mapper utilizes interval sampling to aggressively lessen the buffer allocation and tile size choices. This reduction minimally impacts runtime, as operands are accessed and processed tile by tile. Additionally, ReDas Mapper avoids creating small tiles that would lead to significantly low PE utilization and DRAM access efficiency, further narrowing the search space. Loop dimension and order significantly impact the buffer utilization. To prevent inefficient choices in loop dimension and order, ReDas Mapper generates loop nests based on the tile size and buffer allocation, so that the loop dimension and order are chosen carefully to maximize the buffer utilization. Additionally, when a GEMM workload with the same dimensions appears, ReDas Mapper directly uses the previous choice, eliminating redundant search.

Benefiting from the interval sampling search engine, ReDas Mapper efficiently reduces the search space. For instance, in the case of ResNet-50, the size of the search space is reduced from 2.8×10^{10} to an average of 1923 candidates per GEMM workload. On average, ReDas Mapper requires about

TABLE III
BENCHMARKS

DNN Type	DNN Model	# of Layer	Domain	Abbr.
CNN	ResNet-50	54	Image Classification	RE
	EfficientNet-B0	82	Image Classification	EF
	TinyYOLO-V2	9	Object Detection	TY
	FasterRCNN	46	Object Detection	FR
Transformer	ViT	12	Image Classification	VI
	BERT-Large	24	Machine Translation	BE
RNN	GNMT	16	Machine Translation	GN
	DeepSpeech2	9	Automatic Speech Recognition	DS

0.7 seconds per GEMM workload to estimate runtime and select the optimal configuration and mapping.

V. EVALUATION

A. Experimental Setup

Benchmark Following the MLPerf [25] methodology, we choose a variety of DNN models from domains of image classification, object detection, machine translation and automatic speech recognition. The types of DNNs involved include CNN, RNN, and Transformer. The detailed characteristics of the benchmarks are shown in Table III.

Baseline We use TPUv2, Gemini, Planaria, DyNNamic, and SARA as the evaluation baselines. We faithfully implement these architectures with Verilog HDL. The performance, area, power results are obtained by simulator and Synopsys tools. In particular, TPUv2 supports WS dataflow. Gemini supports both WS and OS dataflow while the systolic array shape does not support reshaping. Planaria supports the coarse-grained reshaping with WS dataflow. DyNNamic supports fine-grained reshaping with OS dataflow. SARA supports both fine-grained reshaping and multiple dataflows.

ReDas Implementation ReDas accelerator is implemented with Verilog HDL and verified through RTL simulations. ReDas is synthesized with Synopsys tools in a 28nm process technology, with SRAMs generated by a memory compiler. The ReDas Mapper is implemented in Python 3.8. It imports the DNN model's Open Neural Network Exchange (ONNX) [26] file and exports the ReDas hardware configuration and workload mapping strategy. The same hardware parameters are used for the above baselines and ReDas for a fair comparison. The reshaping granularity of ReDas is limited to 4×4 , which is consistent with SARA. The detailed parameters are shown in Table IV. We also implement the double-buffered on-chip memories for accelerators. These modifications highlight the impact of dataflow and reshaping capabilities rather than the low-level micro-architecture.

Experimental Methodology We closely follow the description of the original papers to reproduce the baseline hardware architectures and hardware configuration space. Because the detailed workload mapping strategies are unavailable for some baselines in original paper, we construct the GEMM mapping spaces and analytical models for accelerators and search for

TABLE IV
ReDAS CONFIGURATION PARAMETERS

Parameter	Value
Systolic array size	128x128
PE operating frequency	700 MHz
PE data type	Int8
Technology	28nm
On-chip SRAM size	4 MB
Number of memory channels	8
Memory bandwidth	256 GB/S

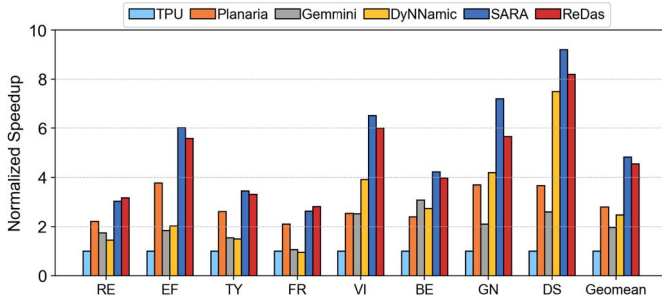


Fig. 11. Performance of ReDAS and other baseline accelerators on eight DNN workloads against the TPU.

configurations with minimal runtime for a fair comparison. Moreover, for the depth-wise convolution layer, following existing implementation [27], we rearrange multiple weight vectors into a large weight matrix, optimizing the PE utilization and performance of systolic array-based accelerators. Following the methodology from prior work [8], [13], [14], [28], we evaluate performance in an extended SCALE-sim-v2 [29], a widely used cycle-accurate simulator for systolic array architecture. The simulator is verified against the Verilog implementation. We generate the Switching Activity Interchange Format (SAIF) files for every accelerator with real input data to estimate power consumption using the Design Compiler. DRAMsim3 [30] is integrated with the simulator to model DRAM behaviors and energy consumption.

B. Performance Analysis

Fig. 11 shows the normalized speedup brought by ReDAS compared to baselines. ReDAS achieves $4.6\times$ speedup on average (geometric mean) against the TPU. For the baseline accelerators that only support multiple dataflow or coarse-grained reshaping, ReDAS achieves $2.31\times$ and $1.62\times$ speedup on average, compared to Gemini and Planaria. For the accelerators that support fine-grained reshaping, ReDAS also achieves a $1.83\times$ speedup over DyNNamic and comparable performance against SARA.

Among all benchmarks, DeepSpeech2, GNMT, and ViT take the most significant benefit from our design, gaining $8.19\times$, $5.66\times$ and $6.01\times$ speedup compared to TPU. On the one hand, the LSTM layers in DeepSpeech2 and GNMT, which tend to be transformed into GEMM operations with at least one dimension

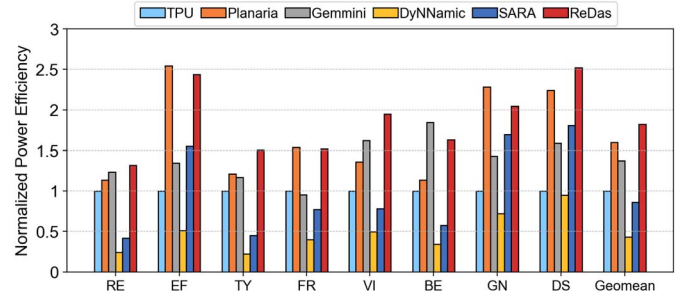


Fig. 12. Power efficiency of ReDAS and other baseline accelerators on eight DNN workloads against the TPU.

significantly smaller than that of the systolic array, are better suited to ReDAS than to the baseline architectures. On the other hand, the feed-forward network (FFN) kernels are the major payload in ViT. They account for 55.1% of the MAC operations. The GEMM dimensions of the FFNs, which are (50, 3072, 768) and (50, 768, 3072), enable ReDAS to achieve a $7.5\times$ speedup when reconfigured to a 52×304 logical shape under OS dataflow.

ReDAS is slightly slower than SARA on GNMT workloads, with SARA achieving a speedup of $1.3\times$ compared to ReDAS. This is because of the numerous irregular but small GEMM workloads in these specific workloads. For instance, the matrix-vector multiplication in GNMT. SARA demonstrates higher performance in handling these workloads due to its shorter setup stage when operating under the parallel PE sub-arrays processing mode. However, such design requires a higher on-chip buffer bandwidth and additional dedicated links for every 4×4 PE array in each direction. As a result, SARA incurs higher power and area overheads. The detailed analysis is in Section V-G.

C. Power Efficiency Analysis

Fig. 12 shows the power efficiency of accelerators on the evaluated workloads. All values are normalized to TPU. ReDAS offers power efficiency improvements ranging from $1.32\times$ to $2.52\times$ compared to TPU. Compared to SARA, ReDAS improves power efficiency, ranging from $1.2\times$ to $3.3\times$, averaging $2.11\times$. This improvement is due to SARA and DyNNamic employing multi-ported buffers for fine-grained reshaping, significantly increasing SRAM read/write energy costs. In contrast, ReDAS uses roundabout data paths and lightweight surrounding buffers, enhancing data reuse among PEs. As a result, ReDAS achieves high power efficiency.

Among all benchmarks, Gemini shows higher power efficiency than ReDAS on BERT-Large workloads. In particular, Gemini achieves a $1.13\times$ improvement in power efficiency over ReDAS. This is because the size of most GEMM operations in BERT-Large is larger than the size of PE array in both dimensions, such as (128, 1024, 4096), (128, 1024, 1024) and (128, 4096, 1024). Therefore, ReDAS tends to operate under the 128×128 logical shape in OS/WS dataflow, similar to Gemini's configuration, but the roundabout data paths in ReDAS consume additional energy.

TABLE V
AREA AND ENERGY BREAKDOWN

Module	Area (mm ²)	Area (%)	Energy (mJ)	Energy (%)
PE array	9.19	44.2	5.21	67.8
MACs	3.76	18.1	1.29	16.8
Original Muxes & Regs ³	2.74	13.2	1.61	20.9
Additional Muxes & Regs	2.69	13.0	2.31	30.0
Multi-mode Buffers	10.21	49.2	1.05	13.7
SRAM Macros	9.08	43.7	0.81	10.5
Accumulators	0.37	1.8	0.03	0.4
Bank Controllers	0.76	3.7	0.21	2.7
SIMD units	0.85	4.1	0.18	2.3
Controller	0.15	0.7	< 0.01	< 0.1
Instruction buffer	0.06	0.3	< 0.01	< 0.1
DMA	0.31	1.5	0.22	2.9
Off-chip memory	-	-	1.01	13.1
Total	20.77	100.0	7.69	100.0

D. Area and Energy Breakdown

The ReDAS is synthesized using Synopsys tools in a 28nm process technology, specifically at the 0.81v@125C corner. The synthesized report shows the critical path takes 0.87 ns, which means ReDAS can run at up to 1.15 GHz.

Table V displays the synthesized area and energy breakdowns of ReDAS for an inference on ResNet-50. Most of the accelerator footprint comes from the PE array and on-chip buffers. ReDAS introduces additional components for fine-grained reshaping and multiple dataflows, such as mux units, registers, and bank controllers. Compared to the TPU architecture, these additional components increase area overhead by 35.3%. Regarding power consumption, the synthesis report shows that the PE array in ReDAS increases the power consumption by 149% on average compared to the PE array in TPU. The power overhead mainly comes from the high PE utilization. Due to the ability of fine-grained reshaping and multiple dataflows, the PE utilization in ReDAS is $4.75\times$ higher than TPU. The more PEs involved in computation, the higher the power consumption is. As a result, ReDAS achieves an improved power efficiency and energy efficiency. Compared to TPU, ReDAS achieves about $1.8\times$ power efficiency, $8.3\times$ energy-delay product (EDP) reduction and $3.4\times$ area-delay product (ADP) reduction.

The distributed multi-mode buffers bring the insignificant overhead of die area and power. The synthesized report shows that the area of ReDAS on-chip buffers is 10.21 mm^2 . For comparison, the TPU-like concentrated buffers are 8.97 mm^2 , and the SARA's multi-ported buffers are 56.47 mm^2 . In the aspect of the buffer power consumption, ReDAS distributed buffer is 4.19 pJ/byte , while the TPU-like concentrated buffer is 3.92 pJ/byte . However, the total energy consumption of SRAM access is roughly the same as TPU, as the roundabout data paths enhance the data reuse among the PEs, reducing the number of

³The Muxes and registers (the stationary and non-stationary data registers) of PE in Gemmini-like systolic array.

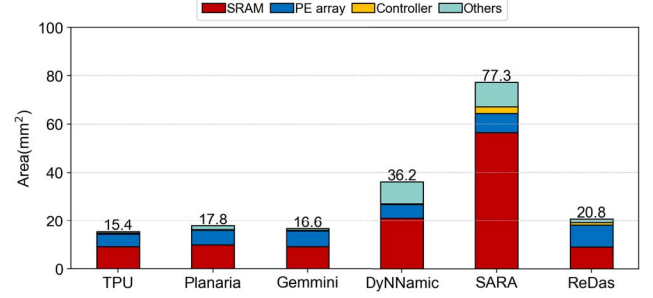


Fig. 13. On-chip Area for ReDAS and other baseline accelerators.

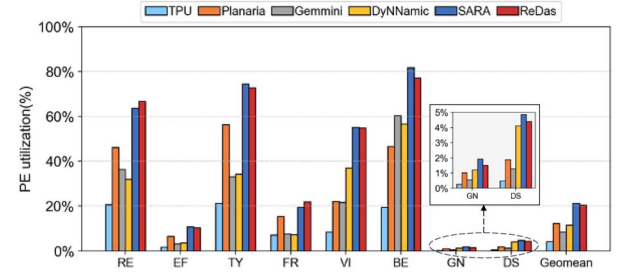


Fig. 14. PE utilization for different DNN models.

SRAM accesses. In the aspect of off-chip memory, ReDAS takes 13.31 pJ/byte on average to access data through HBM2.

Fig. 13 provides an area breakdown comparison between ReDAS and other baseline accelerators. The on-chip area is divided into four parts: SRAM, PE array, Controller, and Other. The on-chip buffers and PE array are the predominant contributors to the area footprint. As for the computing components, the synthesis report shows that the PE array in ReDAS increases the area by 77.4% compared to the TPU-like PE array. The main reason is the additional muxes and registers within the PE array. Although the PE array raises the area overhead, the overall increase of the chip area is 35.3% because the on-chip memory takes up a significant portion of the chip area. When compared to other accelerators such as DyNNamic and SARA, both of which also support fine-grained reshaping, ReDAS demonstrates a much smaller overall area. For instance, ReDAS takes up about 27% of the SARA area. The SRAM is the dominant contributor to the area footprint of SARA and DyNNamic.

E. PE Utilization Analysis

We evaluate the PE utilization of the baseline accelerators and ReDAS by running different DNN models. The PE utilization is defined as the ratio between the average number of activated PEs per cycle and the total number of PEs available in the architecture. The results are shown in Fig. 14. ReDAS can achieve $4.79\times$, $1.67\times$ and $2.42\times$ higher PE utilization over TPU, Planaria and Gemmini on average, respectively. The PE utilizations for GNMT and DeepSpeech2 are significantly lower than for other DNN workloads, primarily because the major operations in these RNN models are matrix-vector multiplication

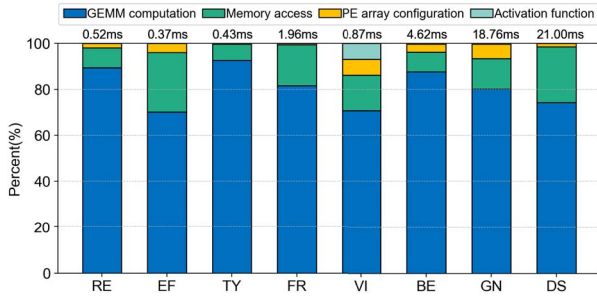


Fig. 15. Latency and runtime breakdown with different DNN workloads.

operations. Moreover, the PE utilizations for EfficientNet-B0 and FasterRCNN, which exploit depth-wise convolutions, are smaller than other CNN models. This is because the depth-wise 2-D filter is vectorized and mapped to a few columns of the array, even when optimized by filter gathering [27].

F. Runtime Breakdown

We further break down the runtime of DNN workloads into four parts, i.e., GEMM computation, Memory access, PE array configuration, and Activation function. As shown in Fig. 15, the execution of GEMMs significantly contributes to the total runtime. Due to the ping-pong work mode of the on-chip buffer, memory access operations are performed with GEMM operations simultaneously. As a result, only about 7% to 25% of the total runtime is non-overlapping. Furthermore, ReDas takes 128 cycles to configure the PE array and multi-mode buffers at the beginning of every GEMM execution, resulting in 0.4% to 7.0% runtime cost. The rest of the execution time is consumed by the non-linear activation and pooling layers, including max pooling, ReLU, sigmoid, softmax, and normalization, resulting in 0.1% to 6.9% runtime cost. We analyzed the cycles caused by the roundabout data path during the GEMM computations, as indicated by the additional bypass cycles of T_{exe} in Equation (4). The results demonstrate that ReDas accounts for an average of 1.2% of the total runtime, indicating minimal impact on performance.

G. EDP and ADP Analysis

To comprehensively evaluate the ReDas architecture, an analysis of the energy-delay product (EDP) and area-delay product (ADP) for each accelerator in the benchmark workloads is conducted. The EDP is calculated as the product of energy consumption and delay, while the ADP is the product of die area and delay. These metrics provide a comprehensive assessment of the trade-off between energy consumption, area cost, and computational efficiency. A lower EDP value indicates a more efficient balance between energy consumption and processing time, while a smaller ADP signifies a better balance between die area and delay.

In Fig. 16, the experimental results for the EDP are presented. The numbers on each colored bar in the figure are normalized to ReDas. On average, ReDas achieves $8.3\times$ reduction in EDP

compared to the TPU architecture. The lightweight design of ReDas contributes to a better balance of energy and performance when compared to SARA. Notably, ReDas exhibits EDP reduction up to $3.3\times$ and $2.0\times$ on average compared to SARA across different benchmarks.

Fig. 17 illustrates the ADP results. On average, ReDas achieves $3.4\times$ reduction in ADP compared to TPU. As discussed in Section II-E, previous designs aimed at supporting fine-grained reshaping and multiple dataflows often encountered high area overhead. However, ReDas successfully addresses this challenge, resulting in reduced ADP compared to DyNNamic and SARA architectures. On average, ReDas achieves an ADP that is 68% and 71% lower than DyNNamic and SARA, respectively. These results demonstrate the lightweight and effective nature of the ReDas architecture.

H. Sensitivity Analysis

We conducted sensitivity experiments to analyze the performance impact of the design points in ReDas architecture.

Hardware design points Fig. 18 illustrates the geometric mean speedup achieved by ReDas when supporting each or both design points, as compared to the TPU architecture. The experiment is conducted across four different PE array scales, ranging from 16×16 to 128×128 .

Fine-grained reshaping and multi-dataflow reconfiguration contribute significant performance improvement, gaining $3.5\times$ and $2.5\times$ speedup with PE array size of 128×128 , respectively. Combining both design points pushes the enhancement even higher to $4.6\times$ speedup, which demonstrates the effectiveness of ReDas's roundabout data path design and multi-dataflow support.

Furthermore, Fig. 18 demonstrates a rising trend on performance improvement of ReDas compared to TPU as PE array size increases. This results from the fine-grained reshaping and multi-dataflow reconfiguration which render ReDas enough flexibility to utilize more PE for various DNN workloads when PE array grows larger. In contrast, TPU and other accelerators with low adaptability suffer from dramatic decrease of PE utilization.

Mapping process Fig. 19 illustrates the mapping time of brute-force searching and interval sampling searching with different DNN workloads. Due to the huge map space, the brute-force search takes several days to several months to generate the ReDas hardware configuration and workload mapping strategy. By contrast, on average, ReDas Mapper's interval sampling engine reduces the mapping time by six orders of magnitude. Regarding DNN workloads' execution time, the results of interval sampling search introduce a range of 0.1% to 2% performance loss compared to the results from brute-force search, showing the effectiveness of ReDas's interval sampling engine.

Dataflow and logical PE array shape Figs. 20 and 21 illustrate the distribution of dataflows and logical shapes during ReDas executing various DNN workloads. ReDas operates approximately 40.9% of DNN layers using OS dataflow and 39.7% using WS dataflow. Concerning logical PE array shapes,

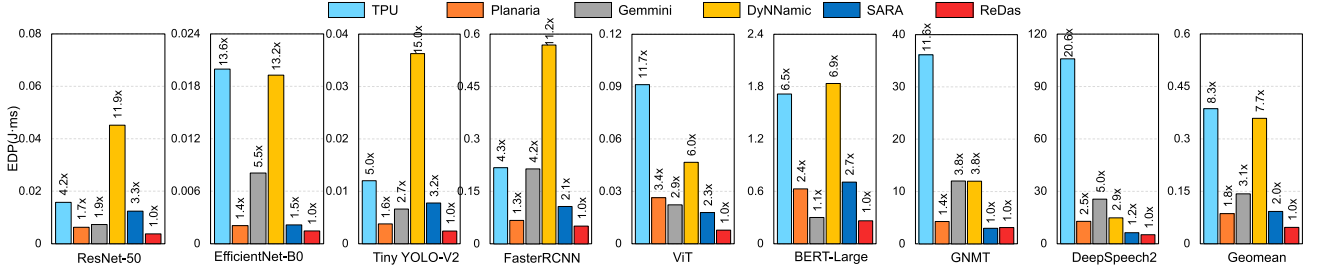


Fig. 16. Energy-delay product (EDP) of accelerators with different DNN workloads.

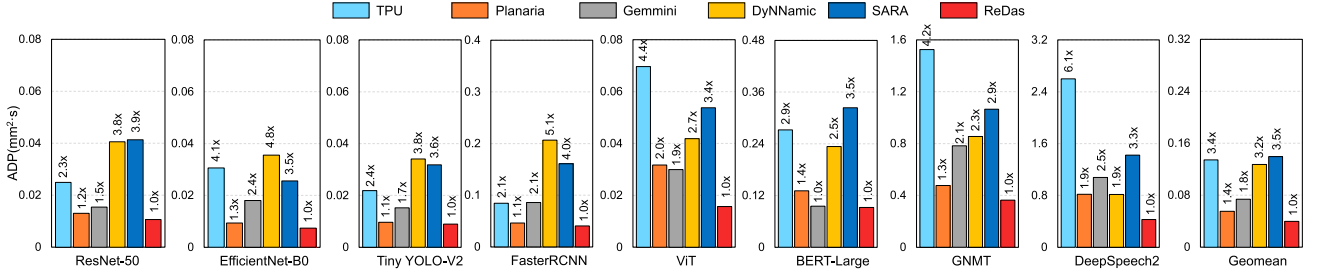


Fig. 17. Area-delay product (ADP) of accelerators with different DNN workloads.

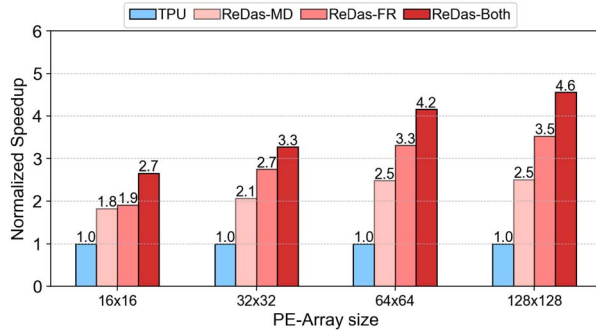


Fig. 18. Normalized performance of TPU and ReDas with different design points at several PE array scales. **ReDas-MD**: Only support dynamic reconfiguration for multiple dataflows. **ReDas-FR**: Only support fine-grained reshaping under WS dataflow. **ReDas-Both**: Support dynamic reconfiguration for multiple dataflows and fine-grained reshaping.

the 256x64 configuration is the most prevalent, representing 27.3% DNN layers. GNMT, BERT-Large, and ResNet-50 are the top three DNNs utilizing this configuration. Other logical shapes are also employed, ranging from 0.2% to 9.2% of DNN layers. These distributions highlight the importance of supporting fine-grained reshaping and multiple dataflows.

We conducted a case study to illustrate how ReDas achieves high performance through fine-grained reshaping and multiple dataflows. In Fig. 22, the runtime of four DNN layers is shown with various logical shapes and dataflows. We utilized ReDas Mapper to generate the remaining configurations for a specific dataflow and logical shape. The yellow triangles highlight the dataflow and logical shape with the minimum runtime. Performance varies almost continuously with different logical shapes, and ReDas achieves optimal performance with

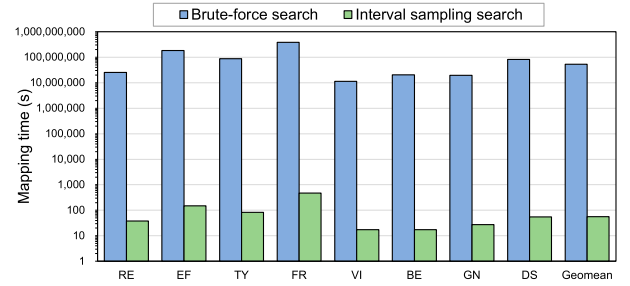


Fig. 19. The mapping time of brute-force searching and interval sampling searching with different DNN workloads.⁴

various configurations across DNN layers. Although the roundabout data paths may not use all the PEs in the systolic array, ReDas can gain performance through fine-grained reshaping. For instance, as depicted in the top-left subfigure in Fig. 22, the GEMM dimension of the second layer of TinyYOLO-V2 is (43264, 32, 144), ReDas achieves optimal performance when reshaping to 384×32 with OS dataflow, where 75% of PEs are involved in GEMM execution. In comparison, when mapping the workload to the conventional systolic array (128x128, OS), only the first 32 columns of the PE array (25% of PEs) are involved in GEMM execution. As a result, the ReDas sized 384×32 achieves 3.79× speedup compared to the ReDas sized 128×128.

⁴The mapping procedures are executed on the Intel Xeon Gold 5218R CPUs operating at 2.1 GHz. The mapping time of the brute-force search is the CPU time consumed by a multi-threaded program.

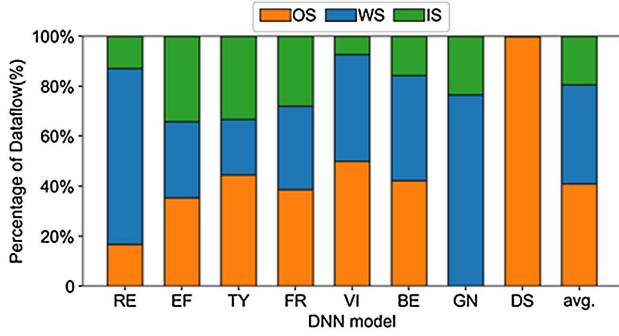


Fig. 20. Dataflow breakdown across DNN workloads.

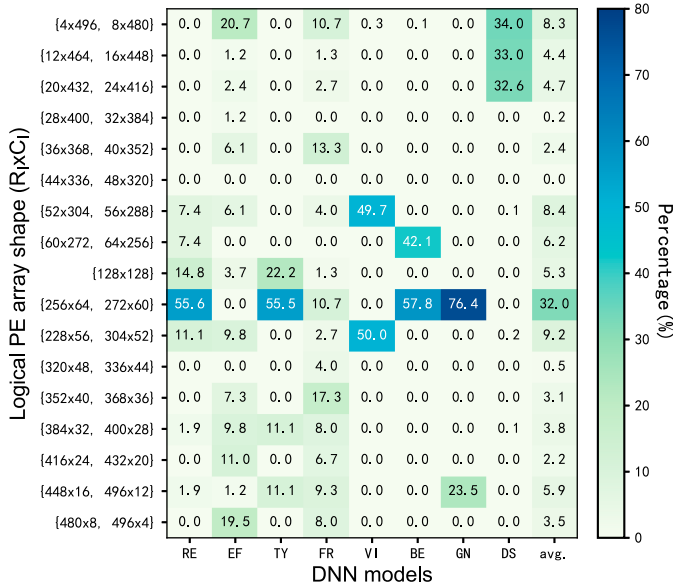


Fig. 21. Heatmap of ReDas's logical PE array shape across different DNN workloads.

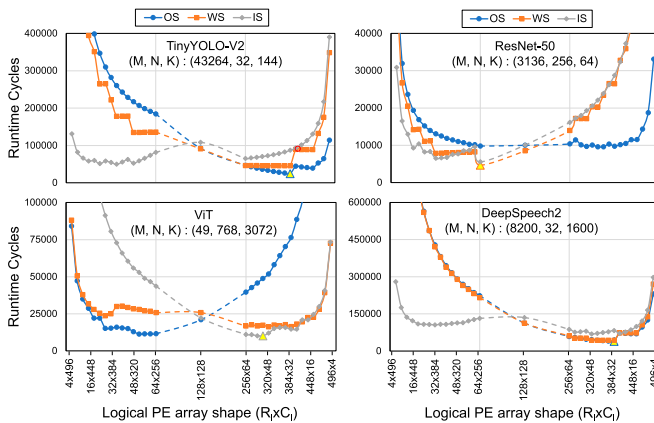


Fig. 22. The runtime of DNN layers with different logical shapes and dataflows.

VI. CONCLUSION

We proposed ReDas, a flexible and lightweight systolic array to support fine-grained reshaping and multiple dataflows. Through the innovative design of the roundabout data path,

the PE structure, and the multi-mode buffer, ReDas is capable of adapting to DNN layers by dynamically reconfiguring the logical shapes and dataflows of the systolic array, achieving this flexibility at a low cost. The evaluation results demonstrate that, in comparison to conventional systolic arrays, ReDas achieves about $4.6\times$ speedup and $8.3\times$ energy-delay product (EDP) reduction. For future work, we aim to explore the performance benefits of accelerating structurally sparse neural networks [31], [32] and to investigate the application of advanced power management techniques, such as dynamic voltage and frequency scaling (DVFS), to further optimize the energy efficiency of the ReDas architecture in various scenarios.

REFERENCES

- [1] L. Alzubaidi et al., "Review of deep learning: Concepts, CNN architectures, challenges, applications, future directions," *J. Big Data*, vol. 8, no. 53, pp. 1–74, Mar. 2021.
- [2] N. P. Jouppi et al., "Ten lessons from three generations shaped Google's TPUv4: Industrial product," in *Proc. ACM/IEEE 48th Annu. Int. Symp. Comput. Archit. (ISCA)*, 2021, pp. 1–14.
- [3] Y.-H. Chen et al., "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, Jan. 2017.
- [4] Y. S. Shao et al., "Simba: Scaling deep-learning inference with multi-chip-module-based architecture," in *Proc. 52nd Annu. IEEE/ACM Int. Symp. Microarchit. (MICRO)*, 2019, pp. 14–27.
- [5] T. Chen et al., "DianNao: A small-footprint high-throughput accelerator for ubiquitous machine-learning," *ACM SIGARCH Comput. Archit. News*, vol. 42, no. 1, pp. 269–284, 2014.
- [6] H. T. Kung and C. E. Leiserson, "Systolic arrays (for VLSI)," in *Proc. Sparse Matrix*, vol. 1, 1979, pp. 256–282.
- [7] E. Qin et al., "SIGMA: A sparse and irregular GEMM accelerator with flexible interconnects for DNN training," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA)*, 2020, pp. 58–70.
- [8] J. Lee et al., "Dataflow mirroring: Architectural support for highly efficient fine-grained spatial multitasking on systolic-array NPIs," in *Proc. 58th ACM/IEEE Des. Automat. Conf. (DAC)*, 2021, pp. 247–252.
- [9] B. Liu et al., "Addressing the issue of processing element under-utilization in general-purpose systolic deep learning accelerators," in *Proc. 24th Asia South Pacific Des. Automat. Conf. (ASP-DAC)*, 2019, pp. 733–738.
- [10] M. Tan and Q. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. Process. Int. Conf. Mach. Learn. (ICML)*, 2019, pp. 6105–6114.
- [11] H. Genc et al., "Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration," in *Proc. 58th ACM/IEEE Des. Automat. Conf. (DAC)*, 2021, pp. 769–774.
- [12] S. Ghodrati et al., "Planaria: Dynamic architecture fission for spatial multi-tenant acceleration of deep neural networks," in *Proc. 53rd Annu. IEEE/ACM Int. Symp. Microarchit. (MICRO)*, 2020, pp. 681–697.
- [13] E. Hanson et al., "DyNNamic: Dynamically reshaping, high data-reuse accelerator for compact DNNs," *IEEE Trans. Comput.*, vol. 72, no. 3, pp. 880–892, Mar. 2023.
- [14] A. Samajdar et al., "Self adaptive reconfigurable arrays (SARA) learning flexible GEMM accelerator configuration and mapping-space using ML," in *Proc. 59th ACM/IEEE Des. Automat. Conf. (DAC)*, 2022, pp. 583–588.
- [15] V. M. Raju, V. Gupta, and S. Lomate, "Performance of open autonomous vehicle platforms: Autoware and Apollo," in *Proc. IEEE 5th Int. Conf. Convergence Technol. (I2CT)*, 2019, pp. 1–5.
- [16] K. Chellapilla, S. Puri, and P. Simard, "High performance convolutional neural networks for document processing," in *Proc. 10th Int. Workshop Frontiers Handwriting Recognit.*, 2006.
- [17] E.-Y. Yang et al., "FlexACC: A programmable accelerator with application-specific ISA for flexible deep neural network inference," in *Proc. IEEE 32nd Int. Conf. Appl.-Specific Syst., Archit. Processors (ASAP)*, 2021, pp. 266–273.
- [18] R. Xu et al., "A survey of design and optimization for systolic array based DNN accelerators," *ACM Comput. Surveys*, vol. 56, no. 1, pp. 1–37, 2023.
- [19] Z.-G. Liu, P. N. Whatmough, and M. Mattina, "Systolic tensor array: An efficient structured-sparse GEMM accelerator for mobile CNN

- inference,” *IEEE Comput. Archit. Lett.*, vol. 19, no. 1, pp. 34–37, Jan.–Jun. 2020.
- [20] K. He et al., “Deep residual learning for image recognition,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.
- [21] Y.-M. Lee, C.-P. Chen, and D. Wong, “Optimal wire-sizing function under the Elmore delay model with bounded wire sizes,” *IEEE Trans. Circuits Syst. I, Fundam. Theory Appl.*, vol. 49, no. 11, pp. 1671–1677, Nov. 2002.
- [22] C. C. Chu and D. Wong, “Closed form solution to simultaneous buffer insertion/sizing and wire sizing,” in *Proc. Int. Symp. Phys. Des.*, 1997, pp. 192–197.
- [23] A. M. Abdelhadi and G. G. Lemieux, “Modular multi-ported SRAM-based memories,” in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays*, 2014, pp. 35–44.
- [24] J. Yu et al., “NN-LUT: Neural approximation of non-linear operations for efficient transformer inference,” in *Proc. 59th ACM/IEEE Des. Automat. Conf. (DAC)*, 2022, pp. 577–582.
- [25] V. J. Reddi et al., “MLPerf inference benchmark,” in *Proc. ACM/IEEE 47th Annu. Int. Symp. Comput. Archit. (ISCA)*, 2020, pp. 446–459.
- [26] “ONNX runtime, version 1.12.0.” ONNX Runtime. Accessed: Jul. 22, 2022. [Online]. Available: <https://onnxruntime.ai/>
- [27] Z. Qin, Z. Zhang, D. Li, Y. Zhang, and Y. Peng, “Diagonalwise refactorization: An efficient training method for depthwise convolutions,” in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Piscataway, NJ, USA: IEEE Press, 2018, pp. 1–8.
- [28] Y. Choi and M. Rhu, “PREMA: A predictive multi-task scheduling algorithm for preemptible neural processing units,” in *Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA)*, 2020, pp. 220–233.
- [29] A. Samajdar et al., “A systematic methodology for characterizing scalability of DNN accelerators using SCALE-Sim,” in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw. (ISPASS)*, 2020, pp. 58–68.
- [30] S. Li et al., “DRAMsim3: A cycle-accurate, thermal-capable DRAM simulator,” *IEEE Comput. Archit. Lett.*, vol. 19, no. 2, pp. 106–109, Jul.–Dec. 2020.
- [31] H. Mao et al., “Exploring the granularity of sparsity in convolutional neural networks,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, 2017, pp. 13–20.
- [32] Y. He, X. Zhang, and J. Sun, “Channel pruning for accelerating very deep neural networks,” in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 1389–1397.



Meng Han received the B.S. degree in computer science from Beijing University of Posts and Telecommunications, Beijing, China, in 2019. He is currently working toward the Ph.D. degree in computer architecture with the School of Computer Science and Engineering, Beihang University, Beijing, China. His research interests include computer architecture, deep learning accelerator and high performance computing.



Liang Wang received the B.Eng. and M.Sc. degrees in electronics engineering from Harbin Institute of Technology, China, in 2011 and 2013 respectively, and the Ph.D. degree in computer science and engineering from The Chinese University of Hong Kong, in 2017. He is currently an Assistant Professor with the School of Computer Science and Engineering, Beihang University, China. He was a Postdoctoral Research Fellow with the Institute of Microelectronics, Tsinghua University from 2018 to 2020. His research interests include computer architecture, heterogeneous computing systems, etc.



Limin Xiao received the B.S. degree in computer science from Tsinghua University, Beijing, China, in 1993, and the M.S. and Ph.D. degrees in computer science from the Institute of Computing, Chinese Academy of Sciences, Beijing, in 1996 and 1998, respectively. He is currently a Professor with Beihang University, Beijing. He has authored more than 260 papers in international journals and conferences. His research interests include computer architecture and system software, high performance computer and server system, system virtualization and cloud computing, big data storage and distributed file system and architecture and technology of intelligent computing chip. He is a CCF Distinguished Member.



Tianhao Cai received the B.S. degree in computer science from Beihang University, Beijing, China, in 2022. He is currently working toward the M.S. degree in computer science and technology with the School of Computer Science and Engineering, Beihang University, Beijing, China. His research interests include computer architecture and deep learning accelerator.



Zeyu Wang received the B.S. degree in computer science and technology from Shandong University, Shandong, China, in 2022. He is currently working toward the M.S. degree in computer science and technology with the School of Computer Science and Engineering, Beihang University, Beijing, China. His research interests include computer architecture and deep learning accelerator.



Xiangrong Xu received the B.S. degree in computer science from Beihang University, Beijing, China, in 2020. He is currently working toward the Ph.D. degree in computer architecture with the School of Computer Science and Engineering, Beihang University, Beijing, China. His research interests include computer architecture and GPU.



Chenhao Zhang received the B.S. degree in Internet of Things engineering from China University of Petroleum (East China), Shandong, China, in 2019. He is currently working toward the Ph.D. degree in computer architecture with the School of Computer Science and Engineering, Beihang University, Beijing, China. His research interests include distributed file systems, storage system, high performance computing, and replica technology.